

Week 1-Day1 Assignment:

Activities

1) Find 10 different model providers and their models released in last 12 months

Ex: OpenAI - 5.5 - Release Date: April 2026 – Cut off Date - Nov 2025 - Cost of tokens - Context Window

Public model

When to use this model: Reasoning !!

2) Read 2-3 articles about how transformer works?

- Goal to understand the self attention mechanism, vector embedding, feed forward

3) Connect with your group learners and connect with their linkedin profiles

Model Providers:

1. GPT-5.5 (OpenAI)

- Latest Version: GPT-5.5 Pro / GPT-5.4-Cyber
- Release Date: April 24 2026 (Pro), Knowledge cutoff Dec 1, 2025
- Notes: This model has improved reasoning, math, and scientific capabilities, with specific versions designed for cybersecurity professionals.

GPT-5.5 Pro is OpenAI's high-capability model optimized for deep reasoning and accuracy on complex, high-stakes workloads. It features a 1M+ token context window (922K input, 128K output) with support for text and image inputs, and is designed for long-horizon problem solving, agentic coding, and precise execution across multi-step workflows.

2. Claude Mythos (Anthropic)

- Latest Version: Claude Mythos Preview

- Release Date: April 7 2026, **Knowledge Cutoff:** Claude Mythos Preview has a knowledge cutoff date of **December 2025**

- Notes: This model leads in reasoning benchmarks, replacing the Claude 4 Opus series as the top frontier model.

Claude Mythos is a high-capability AI frontier model from Anthropic that is **not generally available** to the public due to its advanced, potentially dangerous autonomous cybersecurity capabilities.

3. Gemini 3.1 Pro (Google DeepMind)

- Latest Version: Gemini 3.1 Pro

- Release Date: April 2026 (Preview), The knowledge cutoff for Gemini 3.1 Pro is January 2025

- Notes: This model is optimized for agentic workflows and complex coding tasks, featuring native multimodality.

Designed for advanced development and agentic systems, Gemini 3.1 Pro Preview improves long-horizon stability and tool orchestration while increasing token efficiency. It introduces a new medium thinking level to better balance cost, speed, and performance. The model excels in agentic coding, structured planning, multimodal analysis, and workflow automation, making it well-suited for autonomous agents, financial modeling, spreadsheet automation, and high-context enterprise tasks.

4. Grok-4.1 (xAI)

- Latest Version: Grok 4.1 (Thinking)

- Release Date: January 2026, knowledge cutoff of **July 2025**

- Notes: This model is known for real-time data integration and extended reasoning capabilities.

Key Features: This model includes a 2-million-token context window, multimodal support (text/images), and operates in both reasoning and non-reasoning modes

5. DeepSeek-V4-Pro (DeepSeek-AI)

- Latest Version: DeepSeek-V4-Pro

- Release Date: April 2026, knowledge cutoff in **2025-05**

- Notes: This is a top open-weight model recognized for high-performance coding and creative writing.

Performance: The model is noted for strong performance in reasoning, STEM, and agentic coding workflows, with benchmarks showing it rivals top closed-source models.

6. Kimi K2.6 (Moonshot AI)

- Latest Version: Kimi K2.6
- Release Date: April 19, 2026, knowledge cutoff of **April 2025**
- Notes: This model is highly rated for efficient reasoning and high-speed, low-cost API access. It is designed to be highly proficient in coding, web browsing, and data analysis.

7. Qwen 3.6-Plus (Alibaba Cloud)

- Latest Version: Qwen 3.6-Plus
- Release Date: April 2 2026, The knowledge cutoff for **Qwen 3.6-Plus** is **early 2026**.
- Notes: This is a powerful, open-weights model specialized in Chinese language proficiency, mathematics, and coding. It delivers major gains in agentic coding, front-end development, and overall reasoning, with a significantly improved “vibe coding” experience. The model excels at complex tasks such as 3D scenes, games, and repository-level problem solving, achieving a 78.8 score on SWE-bench Verified. It represents a substantial leap in both pure-text and multimodal capabilities, performing at the level of leading state-of-the-art models.

8. Muse Spark (Meta Superintelligence Labs)

- Latest Version: Muse Spark
- Release Date: April 8 2026, **Cutoff:** It was trained on data from the **early 2026 period**.
- Notes: This model was developed as a successor to the Llama series for advanced, open-source intelligence tasks. It is a multimodal reasoning model (text + image) that supports "tool-use," visual chain-of-thought, and "Contemplating" (multi-agent) modes.

9. Mistral Large 3 (Mistral AI)

- Latest Version: Mistral Large 3
-

- Release Date: December 2, 2025 (Widely used into Q2 2026)

Knowledge Cutoff: While some sources indicate a December 2025 cutoff (coinciding with release), a specific, finalized training data cutoff date was not universally published in initial metadata, leaving it generally understood to be around late 2025.

- Notes: This is a high-performing, open-weight Mixture-of-Experts (MoE) model. The model is designed for advanced reasoning, agentic capabilities, and long-context understanding. It is significantly cheaper than its predecessor, Mistral Large 2, priced around \$0.50 per million input tokens and \$1.50 per million output tokens.

10. GLM-5.1 (Zhipu AI)

- Latest Version: GLM-5.1

- Release Date: April 8 2026, **Training Cutoff Knowledge:** As of its release in April 2026, the model features the latest training data for early 2026,

- Notes: This model is optimized for agentic workflows, often used in complex autonomous systems. **GLM-5.1** is a state-of-the-art open-source Large Language Model (LLM) focused heavily on agentic software engineering and long-horizon tasks. It gained attention for surpassing established models like Claude Opus 4.6 and GPT-5.4 on key coding benchmarks.

Transformer:

How do transformers work?

Neural networks have been the leading method in various AI tasks such as image recognition and NLP since the early 2000s. They consist of layers of interconnected computing nodes, or *neurons*, that mimic the human brain and work together to solve complex problems.

Traditional neural networks that deal with data sequences often use an encoder/decoder architecture pattern. The encoder reads and processes the entire input data sequence, such as an English sentence, and transforms it into a compact mathematical representation. This representation is a summary that captures the essence of the input. Then, the decoder takes this summary and, step by step, generates the output sequence, which could be the same sentence translated into French.

This process happens sequentially, which means that it has to process each word or part of the data one after the other. The process is slow and can lose some finer details over long distances.

Self-attention mechanism

Transformer models modify this process by incorporating something called a *self-attention mechanism*. Instead of processing data in order, the mechanism enables the model to look at different parts of the sequence all at once and determine which parts are most important.

Imagine that you're in a busy room and trying to listen to someone talk. Your brain automatically focuses on their voice while tuning out less important noises. Self-attention enables the model do something similar: it pays more attention to the relevant bits of information and combines them to make better output predictions. This mechanism makes transformers more efficient, enabling them to be trained on larger datasets. It's also more

effective, especially when dealing with long pieces of text where context from far back might influence the meaning of what's coming next.

Input embeddings

This stage converts the input sequence into the mathematical domain that software algorithms understand. At first, the input sequence is broken down into a series of tokens or individual sequence components. For instance, if the input is a sentence, the tokens are words. Embedding then transforms the token sequence into a mathematical vector sequence. The vectors carry semantic and syntax information, represented as numbers, and their attributes are learned during the training process.

You can visualize vectors as a series of coordinates in an n -dimensional space. As a simple example, think of a two-dimensional graph, where x represents the alphanumeric value of the first letter of the word and y represents their categories. The word *banana* has the value (2,2) because it starts with the letter *b* and is in the category *fruit*. The word *mango* has the value (13,2) because it starts with the letter *m* and is also in the category *fruit*. In this way, the vector (x,y) tells the neural network that the words *banana* and *mango* are in the same category.

Now imagine an n -dimensional space with thousands of attributes about any word's grammar, meaning, and use in sentences mapped to a series of numbers. Software can use the numbers to calculate the relationships between words in mathematical terms and understand the human language model. Embeddings provide a way to represent discrete tokens as continuous vectors that the model can process and learn from.

Positional encoding

Positional encoding is a crucial component in the transformer architecture because the model itself doesn't inherently process sequential data in order. The transformer needs a way to consider the order of the tokens in the input sequence. Positional encoding adds information to each token's embedding to indicate its position in the sequence. This is often done by using a set of functions that generate a unique positional signal that is added to the embedding of each token. With positional encoding, the model can preserve the order of the tokens and understand the sequence context.

Transformer block

A typical transformer model has multiple transformer blocks stacked together. Each transformer block has two main components: a multi-head self-attention mechanism and a position-wise feed-forward neural network. The self-attention mechanism enables the model to weigh the importance of different tokens within the sequence. It focuses on relevant parts of the input when making predictions.

For instance, consider the sentences "*Speak no lies*" and "*He lies down*." In both sentences, the meaning of the word *lies* can't be understood without looking at the words next to it. The words *speak* and *down* are essential to understand the correct meaning. Self-attention enables the grouping of relevant tokens for context.

The feed-forward layer has additional components that help the transformer model train and function more efficiently. For example, each transformer block includes:

- Connections around the two main components that act like shortcuts. They enable the flow of information from one part of the network to another, skipping certain operations in between.
- Layer normalization that keeps the numbers—specifically the outputs of different layers in the network—inside a certain range so that the model trains smoothly.
- Linear transformation functions so that the model adjusts values to better perform the task it's being trained on—like document summary as opposed to translation.

Linear and softmax blocks

Ultimately the model needs to make a concrete prediction, such as choosing the next word in a sequence. This is where the linear block comes in. It's another fully connected layer, also known as a dense layer, before the final stage. It performs a learned linear mapping from the vector space to the original input domain. This crucial layer is where the decision-making part of the model takes the complex internal representations and turns them back into specific predictions that you can interpret and use. The output of this layer is a set of scores (often called logits) for each possible token.

The softmax function is the final stage that takes the logit scores and normalizes them into a probability distribution. Each element of the softmax output represents the model's confidence in a particular class or token.

How are transformers different from other neural network architectures?

Recurrent neural networks (RNNs) and convolutional neural networks (CNNs) are other neural networks frequently used in machine learning and deep learning tasks. The following explores their relationships to transformers.

Transformers vs. RNNs

Transformer models and RNNs are both architectures used for processing sequential data. RNNs process data sequences **one** element at a time in cyclic iterations. The process starts with the input layer receiving the first element of the sequence. The information is then passed to a hidden layer, which processes the input and passes the output to the next time step. This output, combined with the next element of the sequence, is fed back into the hidden layer. This cycle repeats for each element in the sequence, with the RNN maintaining a hidden state vector that gets updated at each time step. This process effectively enables the RNN to remember information from past inputs.

In contrast, transformers process entire sequences simultaneously. This parallelization enables much faster training times and the ability to handle much longer sequences than RNNs. The self-attention mechanism in transformers also enables the model to consider the entire data sequence simultaneously. This eliminates the need for recurrence or hidden vectors. Instead, positional encoding maintains information about the position of each element in the sequence.

Transformers have largely superseded RNNs in many applications, especially in NLP tasks, because they can handle long-range dependencies more effectively. They also have greater scalability and efficiency than RNNs. RNNs are still useful in certain contexts, especially where model size and computational efficiency are more critical than capturing long-distance interactions.

Transformers vs. CNNs

CNNs are designed for grid-like data, such as images, where spatial hierarchies and locality are key. They use convolutional layers to apply filters across an input, capturing local patterns through these filtered views. For example, in image processing, initial layers might detect edges or textures, and deeper layers recognize more complex structures like shapes or objects.

Transformers were primarily designed to handle sequential data and couldn't process images. Vision transformer models are now processing images by converting them into a sequential format. However, CNNs continue to remain a highly effective and efficient choice for many practical computer vision applications.